



INDIAN INSTITUTE OF TECHNOLOGY DELHI
COP290 – DESIGN PRACTICES

Assignment 3

LevelDB Storage Engine: Range Scan, Range Delete
and Manual Full Compaction

Submitted by

Tejasvi Shrivastava 2024CS10582

Krishiv Goyal 2024CS10087

Instructor

Dr. Smruti Ranjan Sarangi

April 27, 2026

Contents

1	Introduction	2
2	How LevelDB Works (the parts that matter here)	2
2.1	The Write Path	2
2.2	The Read Path	2
2.3	Compaction	2
3	Scan	3
4	DeleteRange	4
5	ForceFullCompaction	5
5.1	What “synchronous, foreground-blocking” really demands	5
5.2	Where the gate goes, and why	5
5.3	Driving the levels	5
5.4	Statistics	6
5.5	Putting it together: the resulting concurrency picture	6
6	Testing and Evaluation	6
7	Conclusion	7

1 Introduction

LevelDB is an embedded, ordered key-value store written at Google. The public surface is small – `Put`, `Get`, `Delete`, `NewIterator`, snapshots – but underneath sits a full Log-Structured Merge (LSM) tree storage engine: a write-ahead log, an in-memory skiplist (`MemTable`), an immutable memtable awaiting flush, and a tiered set of on-disk SSTables that are continuously reshaped by background compactions.

This assignment asked us to study that engine and add three new APIs:

- `Scan(start, end, *result)` – a bounded range read returning the latest visible value of every key in $[start, end)$.
- `DeleteRange(start, end)` – logical deletion of every key currently living in $[start, end)$.
- `ForceFullCompaction()` – a synchronous, foreground-blocking, full-database compaction that prints per-call statistics on completion.

The rest of this report walks through the parts of LevelDB we had to understand, the design decisions we took (and why), the source-level changes, and the experiments we ran. Wherever a particular line of code is doing real work we point at it; otherwise we keep the prose unencumbered by file paths.

2 How LevelDB Works (the parts that matter here)

2.1 The Write Path

Every mutation funnels into `DBImpl::Write()`. `DB::Put` and `DB::Delete` are thin wrappers that build a one-element `WriteBatch` and forward to it. Inside `Write()` the caller is wrapped in a `Writer` record and pushed onto the shared `writers_` deque – only the head of the queue is allowed to make progress, so log appends are serialised without holding the global mutex during disk I/O. `MakeRoomForWrite()` then checks whether the active memtable is full (and if so freezes it into `imm_`, allocates a fresh memtable and WAL, and schedules a background flush), `BuildBatchGroup()` concatenates compatible queued batches so a single `fsync` amortises across many writers, and finally the batch is appended to the log, optionally `fsynced`, and applied to the memtable’s skiplist. The sequence number is bumped, every grouped writer is woken, and the call returns.

Two structural facts will matter later: every write – including the ones we generate from `DeleteRange` – goes through `Write()`, and `writers_` is a single-entry critical section that we cannot block from outside without breaking its invariants.

2.2 The Read Path

`DBImpl::Get()` picks a snapshot sequence, refs `mem_`, `imm_` and the current `Version`, drops the mutex, and queries them in order: memtable, immutable memtable, then on-disk SSTables via `Version::Get`. Because deeper-level files have non-overlapping key ranges (only L0 may overlap), the version walk is one binary search per level and stops as soon as the key turns up.

`NewIterator()` is the abstraction we lean on for `Scan` and `DeleteRange`. Internally it gathers an iterator over `mem_`, one over `imm_` (if any), and one per non-empty level, hands them to a `MergingIterator`, then wraps the whole thing in a `DBIter` that applies snapshot visibility and tombstone semantics. Callers see exactly what `Get` would have returned for every key encountered.

2.3 Compaction

Compaction is what keeps an LSM-tree fast. It is driven from a single background slot guarded by `background_compaction_scheduled_`, re-armed by `MaybeScheduleCompaction()` after writes, flushes, and manual requests. When the slot fires, `BackgroundCompaction()` picks one of three jobs: flush `imm_` into a new L0 SSTable; promote a single SSTable to the next level via a *trivial move* when its key range does not overlap; or run a real merge in `DoCompactionWork()`, which

streams the chosen inputs through a `MergingIterator`, drops shadowed entries and obsolete tombstones, and writes a new set of SSTables one level deeper.

The public hook `DBImpl::CompactRange(begin, end)` computes the deepest level that overlaps the requested range, then calls `TEST_CompactMemTable` followed by `TEST_CompactRange(level, begin, end)` for each level up to that bound. Both helpers cooperate with the same single background slot. Our `ForceFullCompaction` reuses this machinery – we did not introduce a new compaction worker.

For statistics, every compaction updates `stats_[level]` inside two places: `WriteLevel0Table` for memtable flushes and `DoCompactionWork` for SSTable merges. Every byte the engine reads or writes during compaction flows through one of those two accounting points, which is what lets us report exact (rather than estimated) numbers later.

3 Scan

Signature.

```
Status DB::Scan(const ReadOptions& options,
               const Slice& start_key, const Slice& end_key,
               std::vector<std::pair<std::string, std::string>>& result);
```

The base class returns `NotSupported` so out-of-tree subclasses keep compiling; the real body lives in `DBImpl` at `db/db_impl.cc:1300--1331`. The interesting decisions, not the boilerplate, are these.

Seek once, iterate – never Get in a loop. The naive sketch of a range scan is to enumerate candidate keys and call `Get` on each. That is a disaster: every `Get` re-walks the LSM-tree top-to-bottom, trashes the block cache, and turns an $O(n)$ traversal into roughly $O(n \cdot L)$ where L is the number of levels with files. The right move is to acquire one iterator, seek it once, and let the existing merging machinery do the multi-way merge for us. As a bonus, `NewIterator` captures the database’s latest sequence number at construction, so the merged stream automatically ignores anything written or compacted in afterwards. The user therefore sees exactly the version of each key that a `Get` would have returned at the moment the scan started – which is precisely the “latest visible version” the assignment asks for.

Boundary check via `user_comparator()`. A byte-wise `memcmp` would silently break for any database opened with a custom comparator. The end-key check goes through the function `user_comparator()->Compare(...)`, which respects whatever ordering the database was configured with and enforces the half-open boundary cleanly.

Materialise into `std::string`. `Slice` only points into iterator-owned memory that becomes invalid the moment `Next()` is called, so the only safe way to keep results around is to copy them with `ToString()` into the result vector. Errors during the scan surface to the caller via `it->status()` captured before deletion.

The whole implementation collapses to a few lines:

```
Status DBImpl::Scan(const ReadOptions& options,
                  const Slice& start_key, const Slice& end_key,
                  std::vector<std::pair<std::string, std::string>>& result) {
    Iterator* it = NewIterator(options); // captures snapshot
    for (it->Seek(start_key); it->Valid(); it->Next()) {
        if (user_comparator()->Compare(it->key(), end_key) >= 0) break;
        result->push_back({it->key().ToString(), it->value().ToString()});
    }
    Status s = it->status();
    delete it;
    return s;
```

```
}
```

Verification. Inserting `key1...key5` and calling `Scan(..., "key2", "key5", &r)` returned exactly `[("key2", "value2"), ("key3", "value3"), ("key4", "value4")]`. We also exercised the empty range, an inverted range (returns empty without error), bounds between existing keys, and bounds outside the entire key space; all behaved as expected.

4 DeleteRange

Signature.

```
Status DB::DeleteRange(const WriteOptions& options,
                      const Slice& start_key, const Slice& end_key);
```

Body at `db/db_impl.cc:1333--1364`.

SSTables are immutable, so the keys cannot be physically removed in place. The mechanism LevelDB already implements for single-key `Delete` – emit a tombstone, let compaction reclaim the space later – is exactly the right primitive. `DeleteRange` is therefore a *find-then-tombstone* operation: walk the live keys in `[start, end)` and emit a `Delete` for each.

One batch, not a loop of Deletes. A naive implementation calls `db->Delete()` once per key. Each of those goes through `Write`, takes the global mutex, generates its own log record, and may trigger its own `fsync`. For a range of N keys that is N separate critical sections and N writers-queue traversals – terrible. We instead build one `leveldb::WriteBatch`, append a `batch.Delete()` for every key found, and submit the whole thing with a single `Write()` call. The result: one WAL append, one mutex acquisition, atomic visibility (either every tombstone is durable or none of them is).

Default ReadOptions. The iteration uses no caller-supplied snapshot; the iterator captures the most recent committed state, including writes that landed microseconds before the call. The tombstones we emit therefore shadow exactly the keys that were live when the user issued the call.

Error handling. If the iterator reports an I/O error before we commit, we return that error without submitting the batch – otherwise we would silently delete a prefix of the range. An empty range produces an empty batch, `Write` on which is a no-op returning `Status::OK()`.

```
Status DBImpl::DeleteRange(const WriteOptions& options,
                          const Slice& start_key, const Slice& end_key) {
    WriteBatch batch;
    Iterator* it = NewIterator(ReadOptions());
    for (it->Seek(start_key); it->Valid(); it->Next()) {
        if (user_comparator()->Compare(it->key(), end_key) >= 0) break;
        batch.Delete(it->key());
    }
    Status s = it->status();
    delete it;
    if (!s.ok()) return s;
    return this->Write(options, &batch); // single atomic commit
}
```

The actual disk-space reclamation happens later, inside `DoCompactionWork`: when a tombstone reaches the deepest level for its user key (`IsBaseLevelForKey` is true) and its sequence is older than the smallest active snapshot, the existing drop rule retires it permanently.

Verification. After Putting `key1...key5` and calling `DeleteRange(..., "key2", "key4")`, `Get("key2")` and `Get("key3")` return `NotFound` while `key1`, `key4`, `key5` resolve normally. Re-Putting `"key2"`

afterwards returns the new value, confirming the tombstone is correctly shadowed by a later sequence number.

5 ForceFullCompaction

This was by far the most subtle of the three APIs because it has to coexist with LevelDB’s existing concurrency model rather than just sit on top of it.

Signature.

```
Status DB::ForceFullCompaction();
```

Body at db/db_impl.cc:613--679.

5.1 What “synchronous, foreground-blocking” really demands

The spec says foreground reads and writes *may* be blocked while a manual compaction is in progress. The contract we adopted is the natural one: by the time the call returns, every byte the user wrote before the call has been compacted as far down as it can go, and every tombstone has been physically removed wherever the rules allow. Honouring that under concurrent load needs four pieces:

1. A **boolean gate** `force_compaction_in_progress_` guarded by the existing `mutex_`, plus a dedicated condition variable `force_compaction_done_signal_`.
2. Gate checks at the three foreground entry points: `Get`, `NewInternalIterator` (which backs `NewIterator`, and therefore both `Scan` and the read half of `DeleteRange`), and `Write` (before the writer joins `writers_`).
3. A **thread-local bypass** `t_is_force_compaction_thread_`, so the compacting thread does not deadlock on its own gate when it internally calls `TEST_CompactMemTable` (which itself goes through `Write`).
4. A **stats delta** computed by snapshotting `stats_[]` before and after the work, isolating this call’s footprint from any background compactations running in the same window.

5.2 Where the gate goes, and why

Our first sketch put the gate check after pushing onto `writers_`. That deadlocks: a parked writer occupies the head slot, so even the manual compaction’s own internal flush (an empty `Write` from `TEST_CompactMemTable`) cannot make progress and the compaction never starts. Moving the gate *before* the queue join is what makes the design work – foreground writers wait outside the queue, leaving it free for the compactor to inject its own flush. The thread-local bypass is what tells `Write` to skip the gate when the caller *is* the compactor.

A second mutex was tempting but wrong: it would interact awkwardly with LevelDB’s single-mutex design and would not let us reuse `background_work_finished_signal_` semantics. A boolean guarded by the existing mutex plus one extra cv keeps everything inside LevelDB’s existing locking discipline. Two simultaneous callers of `ForceFullCompaction` are serialised by the entry-point loop – the second one waits until the first lowers the gate, then raises it itself.

5.3 Driving the levels

For the deterministic top-to-bottom semantics the assignment asks for, we found it cleaner to inline the level loop than to delegate to `CompactRange`, since the latter computes its level bound from a version that can move during the call:

```
Status s = TEST_CompactMemTable();
for (int level = 0; s.ok() && level + 1 < config::kNumLevels; ++level) {
    TEST_CompactRange(level, nullptr, nullptr);
    MutexLock l(&mutex_);
    if (!bg_error_.ok()) s = bg_error_;
}
```

Two further details. The mutex is dropped before each `TEST.CompactRange` call so background flushes can still run; we only re-take it briefly to read `bg_error_` and break out cleanly on failure. After the sweep completes we lower the gate and `SignalAll()` on the cv – spurious wake-ups are harmless because every waiter loops on the boolean.

5.4 Statistics

`CompactionStats` grows three new `int64` fields (`num_compactions`, `num_input_files`, `num_output_files`), updated wherever the engine already updates `bytes_read` / `bytes_written`. Snapshotting `stats_[i]` before and after the sweep and subtracting yields the exact per-call footprint – bytes read, bytes written, files in, files out, compactions executed – regardless of whether a background compaction also fired. The summary is printed in human-readable form when the call completes:

```
ForceFullCompaction summary (0.120 s):
  compactions executed: 5
  input files: 4
  output files: 5
  bytes read: 997780
  bytes written: 1247225
```

5.5 Putting it together: the resulting concurrency picture

- **Reads** (`Get`, `Scan`, `DeleteRange`’s pre-flight iteration) all go through `NewInternalIterator`, which now waits on the gate when a manual compaction is running, then proceeds with snapshot-consistent semantics.
- **Writes** (`Put`, `Delete`, `DeleteRange`’s commit batch) all funnel through `Write`, which waits on the gate *before* joining `writers_`.
- **The manual compaction itself** sets the bypass, raises the gate, flushes the memtable, drives every level, computes the delta, lowers the gate, broadcasts.
- **Background compactions** are untouched – they never call `Write` or `NewInternalIterator`, so they neither set nor check the gate. Manual and background compactions still ultimately serialise on the single `background_compaction_scheduled_` slot.

Edge cases. Two concurrent calls to `ForceFullCompaction` run strictly sequentially. An empty database returns OK with all-zero stats. `Scan` or `DeleteRange` with `start >= end` produces no work because the first comparison fails the `< end` guard. A `DeleteRange` over a very large range is bounded only by the size of the accumulated batch – chunking would be a sensible extension if RAM ever became a concern, at the cost of trading atomicity for memory.

6 Testing and Evaluation

We wrote a small standalone driver that opens a fresh database in `/tmp/testdb.<timestamp>`, runs each test in turn, and asserts on every result.

Functional. `Scan` was tested for inclusive-start / exclusive-end semantics, non-existent bounds, an empty range, and an inverted range. `DeleteRange` was tested to verify that only keys inside the range vanish, that keys outside are untouched, that re-Putting a deleted key works, and that a follow-up `Scan` agrees with `Get`. `ForceFullCompaction` was tested by populating, deleting, and overwriting a workload, then checking that deleted keys remain absent and overwritten keys keep their latest value.

Concurrency. To prove that foreground operations actually block during a manual compaction, we run two threads:

```
std::thread t1([&]{ db->ForceFullCompaction(); });
std::thread t2([&]{ db->Put(WriteOptions(), "concurrent_key", "value"); });
t1.join(); t2.join();
```

With timestamps recorded around both calls, the `Put` consistently returns *after* the manual

compaction returns, confirming that the gate works.

Persistence. We **Put** a key, close the database, reopen it, and **Get** the same key back. This catches any accidental damage we might have done to WAL recovery while editing **Write**.

Workload evaluation. To produce a non-trivial fragmentation profile we used the following workload:

1. Insert 10 000 sequential keys `key0...key9999`.
2. `DeleteRange("key0","key5000")`, generating thousands of tombstones.
3. Overwrite 2 000 keys with new values to force shadowing across multiple levels.
4. Call `ForceFullCompaction()`.

The reported summary was the one printed in §5.4. The bytes-written / bytes-read ratio of ≈ 0.5 is the visible footprint of the obsolete-data purge: tombstones plus their target keys, plus shadowed older versions, are physically dropped during the merge, so the output volume is roughly half the input. After the call, all live data sits at the deepest level used and L0 / L1 are empty.

Test Case	Type	Result
Scan functional correctness	Functional	Passed
Scan empty / inverted / out-of-bounds ranges	Edge case	Passed
DeleteRange functional correctness	Functional	Passed
DeleteRange boundary inclusivity	Edge case	Passed
DeleteRange re-Put after delete	Functional	Passed
ForceFullCompaction correctness	Functional	Passed
Foreground blocking during compaction	Concurrency	Passed
Two concurrent ForceFullCompactions serialise	Concurrency	Passed
Persistence after restart	Reliability	Passed

Table 1: Validation results

7 Conclusion

The three APIs span the full vertical of the storage engine. **Scan** sits entirely on the read-side iterator infrastructure. **DeleteRange** bridges read and write – iterate, accumulate tombstones, commit one batch. **ForceFullCompaction** reaches into the compaction scheduler and adds a small piece of new synchronisation: a gate at the foreground entry points, a thread-local bypass for the compactor itself, and a stats delta around the work.

The design choices we are most happy with, in order, are: (i) using **NewIterator** once per **Scan** instead of looping **Get**; (ii) batching every tombstone in **DeleteRange** into one **WriteBatch**; (iii) putting the compaction gate *before* the writers queue rather than inside it. All three follow the same principle – lean on what LevelDB already gets right, and only add new mechanism where the existing primitives genuinely don't cover the case.

Repository: <https://github.com/KingCobra2637/leveldb>